

Taller de Kickoff de Desarrollo

Duración: 2 horas · Grupo de 2 alumnos

#	Bloque	Tiempo	Entregable
1	Stack + Hardening	45 min	Decisiones técnicas, controles de SO y BD identificados
2	Mapa de Trabajo	20 min	Tabla de módulos con responsable y complejidad
3	Protocolo de Nomenclatura	15 min	Documento de convenciones del proyecto
4	Análisis de Requerimientos	40 min	Ficha completa de 3 módulos elegidos

ENTREGABLES AL FINAL: (1) Stack + controles de hardening identificados · (2) Mapa de trabajo · (3) Protocolo de nomenclatura · (4) Análisis de 3 módulos · (5) GIT repo creado y compartido con el profesor

CONTEXTO: El sistema HIPS corre sobre Rocky Linux (última versión estable) con al menos 10 controles de Hardening. La BD PostgreSQL debe cumplir 7 prácticas CIS. Las pruebas deben ser lo más automatizadas posible.

Bloque 1 — Stack Tecnológico y Hardening

 **45 minutos** · Decisiones técnicas del proyecto e identificación de los controles de seguridad a implementar en el sistema operativo y la base de datos.

1.1 Decisiones del stack

Para cada componente, elijan una opción y escriban la justificación. No alcanza con 'es lo que conocemos'.

Componente	Opciones sugeridas	Elección + Justificación del grupo
Lenguaje principal	Python 3.x · Bash	
		← escribir aquí
Web framework	Flask · Django · FastAPI	
		← escribir aquí

1.2 Hardening del Sistema Operativo — Rocky Linux

El servidor HIPS debe tener al menos 10 controles de hardening bien definidos y justificados. Identifiquen aquí cuáles van a implementar, qué configura cada uno y cómo se verifica.

REFERENCIA: CIS Benchmark for Rocky Linux · STIGs de DISA · guías de hardening del SELinux y firewalld. Cada control debe poder verificarse con un comando concreto.

#	Área / Control	Descripción de lo que se configura	Comando de verificación o implementación
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			

Áreas sugeridas (no son las únicas): SELinux enforcing · firewalld · SSH (deshabilitar root, port, claves) · usuarios y privilegios mínimos · auditd configurado · montaje /tmp noexec · banner de login.

1.3 Hardening de la Base de Datos — PostgreSQL (CIS)

La base de datos debe cumplir al menos 7 controles de hardening basados en el CIS PostgreSQL Benchmark. Identifiquen cuáles van a implementar y cómo se verifica cada uno.

REFERENCIA: CIS Benchmark for PostgreSQL · documentación oficial de PostgreSQL security.
Los controles deben ser verificables con comandos SQL o de sistema.

#	Control CIS / Área	Descripción de lo que se configura	Verificación (SQL o comando)
1			
2			
3			
4			
5			
6			
7			

Controles sugeridos (no son los únicos): usuario de aplicación sin superusuario · SSL activo (ssl=on) · log_connections y log_disconnections · contraseñas de usuarios nunca en código · listen_addresses restringido · parámetro password_encryption.

1.4 Esquema inicial de base de datos

Definan las tablas mínimas necesarias antes de programar. Completen las columnas y agreguen las que falten.

Tabla	Columnas mínimas (editar/agregar)	Propósito
alarmas	id, timestamp, tipo_alarma, ip_origen, modulo, resuelta	Registro de todas las alarmas detectadas
acciones_preencion	id, alarma_id, accion, timestamp, resultado	Log del módulo de prevención
usuarios_web	id, username, password_hash, rol, ultimo_login	Acceso a la interfaz web
configuracion_modulos	id, modulo, parametro, valor, activo	Umbrales y config de cada módulo
		← tabla adicional
		← tabla adicional

Bloque 2 — Mapa de Trabajo

 **20 minutos** · Completen la tabla asignando responsable y complejidad a cada módulo. Deben cubrir los 10 módulos de detección + prevención + infraestructura.

2.1 Módulos del sistema

#	Módulo / Componente	Responsable	Complejidad (A/M/B)	Dependencias	Semana
i	Integridad de archivos (/etc/passwd · /etc/shadow · binarios)				
ii	Usuarios conectados (who / last / origen de conexión)				
iii	Sniffers y modo promiscuo (tcpdump · wireshark · ethereal)				
iv	Análisis de logs (/var/log/secure · httpd/access.log · maillog)				
v	Cola de correo (mailq · detección de spam masivo)				
vi	Procesos con alto consumo de recursos (CPU / RAM)				
vii	Directorio /tmp (procesos · scripts ejecutables)				
viii	Ataques DDoS (log DNS provisto por el profesor)				
ix	Archivos cron sospechosos (/etc/crontab · /var/spool/cron)				
x	Intentos de acceso inválidos (brute force · credential stuffing)				
INFRAESTRUCTURA					
—	Módulo de Prevención (30% nota)		ALTA	todos	
—	Interfaz web + dashboard			PostgreSQL	
—	Sistema de alertas por email + dashboard			web, alarmas	
—	Logger central (/var/log/hips/)			todos	

#	Módulo / Componente	Responsable	Complejidad (A/M/B)	Dependencias	Semana
—	PostgreSQL + 7 CIS controls			—	
—	Carpeta encriptada de configuración			—	
—	Rocky Linux + 10 Hardening controls			—	
—	Suite de pruebas automatizadas			todos	
—	Manual de uso + manual de instalación			—	

Referencia de complejidad: A = Alta (>2 días) · M = Media (1–2 días) · B = Baja (<1 día)

Bloque 3 — Protocolo de Nomenclatura

 **15 minutos** · Decidan y documenten las convenciones del proyecto. Una vez acordadas, TODOS siguen estas reglas.

3.1 Nombre del proyecto

Ítem	Decisión del grupo
Nombre del proyecto (código)	ej: hips_rocky, sentinel_hips
Prefijo del sistema (logs, vars)	ej: HIPS_, hips.

3.2 Archivos y carpetas

Ítem	Convención elegida	Ejemplo
Módulos Python	snake_case · kebab-case	module_sniffer.py
Carpetas	snake_case · kebab-case	detection/, prevention/
Archivos de config	snake_case · punto	settings.env, hips.conf
Templates HTML	kebab-case	dashboard-alerts.html
		← agregar si necesitan

3.3 Código Python

Ítem	Convención elegida	Ejemplo
Funciones	snake_case	detect_sniffer(), block_ip()
Clases	PascalCase	SnifferDetector, LogAnalyzer
Constantes	UPPER_SNAKE	MAX_FAILED_ATTEMPTS = 5
Variables de entorno	UPPER_SNAKE con prefijo	HIPS_DB_PASSWORD
Parámetros de módulo	lower con prefijo	sniffer_check_interval = 60

3.4 Base de datos (PostgreSQL)

Ítem	Convención elegida	Ejemplo
Tablas	snake_case · plural	alarmas, acciones_preencion
Columnas	snake_case	ip_origen, tipo_alarma
Índices	idx_tabla_columna	idx_alarmas_timestamp
Claves foráneas	fk_tabla_ref	fk_acciones_alarma

Ítem	Convención elegida	Ejemplo
Usuario de aplicación	nombre_restrictivo	hips_app (sin superusuario)

3.5 GIT — Ramas y commits

Ítem	Convención elegida
Rama principal	main · master
Ramas de módulos	feat/modulo-nombre ej: feat/sniffer-detection
Ramas de fix	fix/descripcion ej: fix/log-format
Formato de commit	tipo(módulo): descripción ej: feat(sniffer): detección promiscuo
Tipos de commit	feat · fix · docs · test · refactor · chore
Frecuencia mínima de commit	Al terminar cada función · Al fin de cada sesión

3.6 Tipos de alarma (log)

Definan un nombre estandarizado para cada tipo de alarma que aparecerá en alarmas.log:

Módulo	Nombre elegido para el tipo de alarma	Ejemplo en log
i	MODIFICACION_ARCHIVO	29/05/2026 :: MODIFICACION_ARCHIVO :: N/A
ii	USUARIO_SOSPECHOSO	29/05/2026 :: USUARIO_SOSPECHOSO :: 192.168.1.10
iii	SNIFFER_DETECTADO	29/05/2026 :: SNIFFER_DETECTADO :: N/A
iv	FAILED_LOGIN_MULTIPLE · SCANNER_HTTP	29/05/2026 :: FAILED_LOGIN_MULTIPLE :: 10.0.0.25
v	MAIL_QUEUE_ALTA	29/05/2026 :: MAIL_QUEUE_ALTA :: N/A
vi	PROCESO_ALTO_CONSUMO	29/05/2026 :: PROCESO_ALTO_CONSUMO :: N/A
vii	ARCHIVO_TMP_SOSPECHOSO	29/05/2026 :: ARCHIVO_TMP_SOSPECHOSO :: N/A
viii	DDOS_DETECTADO	29/05/2026 :: DDOS_DETECTADO :: 203.0.113.5
ix	CRON_SOSPECHOSO	29/05/2026 :: CRON_SOSPECHOSO :: N/A
x	ACCESO_INVALIDO_REPETIDO · CREDENTIAL_STUFFING	29/05/2026 :: ACCESO_INVALIDO_REPETIDO :: 10.0.0.8

Bloque 4 — Análisis de Requerimientos

 **40 minutos** · Completen la ficha de requerimientos para 3 módulos. Elijan los módulos con los que van a arrancar a programar. Deben ser 3 módulos distintos y al menos uno debe ser de los más complejos.

SUGERENCIA: Módulo iii (sniffer — acotado), Módulo iv (logs — el más complejo), Módulo x (accesos inválidos — requiere decisiones de umbral). Son los tres que generan más decisiones de arquitectura.

Módulo ____ — (completar)

Campo	Completar
Nombre del módulo	Módulo ____:
Objetivo concreto	¿Qué amenaza detecta este módulo?
Fuentes de datos	¿Qué archivos, comandos o APIs lee?
Condición de alarma	¿Cuándo exactamente se dispara? Ser específico (ej: >5 intentos en 10 min)
Comportamiento NORMAL (no alarmar)	¿Cuándo puede verse igual pero NO es intrusión?
Comportamiento ANÓMALO (alarmar)	¿Cuándo SÍ es una intrusión o amenaza?
Parámetros configurables	¿Qué umbrales se pueden ajustar desde la interfaz web?
Lógica de detección (pseudocódigo)	1. 2. 3.
Acción de prevención	¿Qué hace el módulo de prevención al detectar la alarma?
Tipo de alarma en log	NOMBRE_ALARMA (según protocolo de nomenclatura)
Contenido del email al admin	Asunto: [HIPS ALERTA] ... Cuerpo: ...
Visibilidad en dashboard	timestamp tipo ip_origen módulo acción_tomada
Casos borde / excepciones	¿Qué situaciones podrían generar falsos positivos?
Test automatizable	¿Cómo se puede simular la condición para probar el módulo?

Módulo ____ — (completar)

Campo	Completar
Nombre del módulo	Módulo ____:
Objetivo concreto	¿Qué amenaza detecta este módulo?

Campo	Completar
Fuentes de datos	¿Qué archivos, comandos o APIs lee?
Condición de alarma	¿Cuándo exactamente se dispara? Ser específico (ej: >5 intentos en 10 min)
Comportamiento NORMAL (no alarmar)	¿Cuándo puede verse igual pero NO es intrusión?
Comportamiento ANÓMALO (alarmar)	¿Cuándo SÍ es una intrusión o amenaza?
Parámetros configurables	¿Qué umbrales se pueden ajustar desde la interfaz web?
Lógica de detección (pseudocódigo)	1. 2. 3.
Acción de prevención	¿Qué hace el módulo de prevención al detectar la alarma?
Tipo de alarma en log	NOMBRE_ALARMA (según protocolo de nomenclatura)
Contenido del email al admin	Asunto: [HIPS ALERTA] ... Cuerpo: ...
Visibilidad en dashboard	timestamp tipo ip_origen módulo acción_tomada
Casos borde / excepciones	¿Qué situaciones podrían generar falsos positivos?
Test automatizable	¿Cómo se puede simular la condición para probar el módulo?

Módulo ____ — (completar)

Campo	Completar
Nombre del módulo	Módulo ____:
Objetivo concreto	¿Qué amenaza detecta este módulo?
Fuentes de datos	¿Qué archivos, comandos o APIs lee?
Condición de alarma	¿Cuándo exactamente se dispara? Ser específico (ej: >5 intentos en 10 min)
Comportamiento NORMAL (no alarmar)	¿Cuándo puede verse igual pero NO es intrusión?
Comportamiento ANÓMALO (alarmar)	¿Cuándo SÍ es una intrusión o amenaza?
Parámetros configurables	¿Qué umbrales se pueden ajustar desde la interfaz web?
Lógica de detección (pseudocódigo)	1. 2. 3.
Acción de prevención	¿Qué hace el módulo de prevención al detectar la alarma?
Tipo de alarma en log	NOMBRE_ALARMA (según protocolo de nomenclatura)
Contenido del email al admin	Asunto: [HIPS ALERTA] ... Cuerpo: ...

Campo	Completar
Visibilidad en dashboard	timestamp tipo ip_origen módulo acción_tomada
Casos borde / excepciones	¿Qué situaciones podrían generar falsos positivos?
Test automatizable	¿Cómo se puede simular la condición para probar el módulo?

Referencia — Estructura de Carpetas Sugerida

Esta es una estructura de referencia. Pueden adaptarla. Lo importante es que todos los archivos tengan un lugar definido ANTES de empezar a programar.

```
hips-project/
├── detection/                # Un archivo por módulo (i-x)
│   ├── file_integrity.py    # Módulo i
│   ├── users_monitor.py     # Módulo ii
│   ├── sniffer_detect.py    # Módulo iii
│   ├── log_analyzer.py      # Módulo iv
│   ├── mail_queue.py        # Módulo v
│   ├── process_monitor.py   # Módulo vi
│   ├── tmp_monitor.py       # Módulo vii
│   ├── ddos_detect.py       # Módulo viii
│   ├── cron_monitor.py      # Módulo ix
│   └── access_monitor.py    # Módulo x
├── prevention/
│   ├── firewall.py          # iptables
│   ├── user_actions.py      # lock/passwd
│   ├── process_kill.py      # kill -9
│   └── service_mgmt.py      # systemctl stop
├── alerts/
│   ├── logger.py            # Escribe /var/log/hips/
│   └── mailer.py            # smtplib
├── web/                      # Flask app
│   ├── app.py
│   ├── auth.py
│   ├── templates/
│   └── static/
├── db/
│   ├── models.py            # Schema PostgreSQL
│   └── migrations/
├── config/                   # Carpeta encriptada
│   └── baseline_hashes.enc
├── tests/                    # pytest
│   └── test_*.py
├── docs/
│   ├── manual_usuario.pdf
│   └── manual_instalacion.pdf
├── .env.example              # Template sin valores reales
├── requirements.txt
└── README.md
```